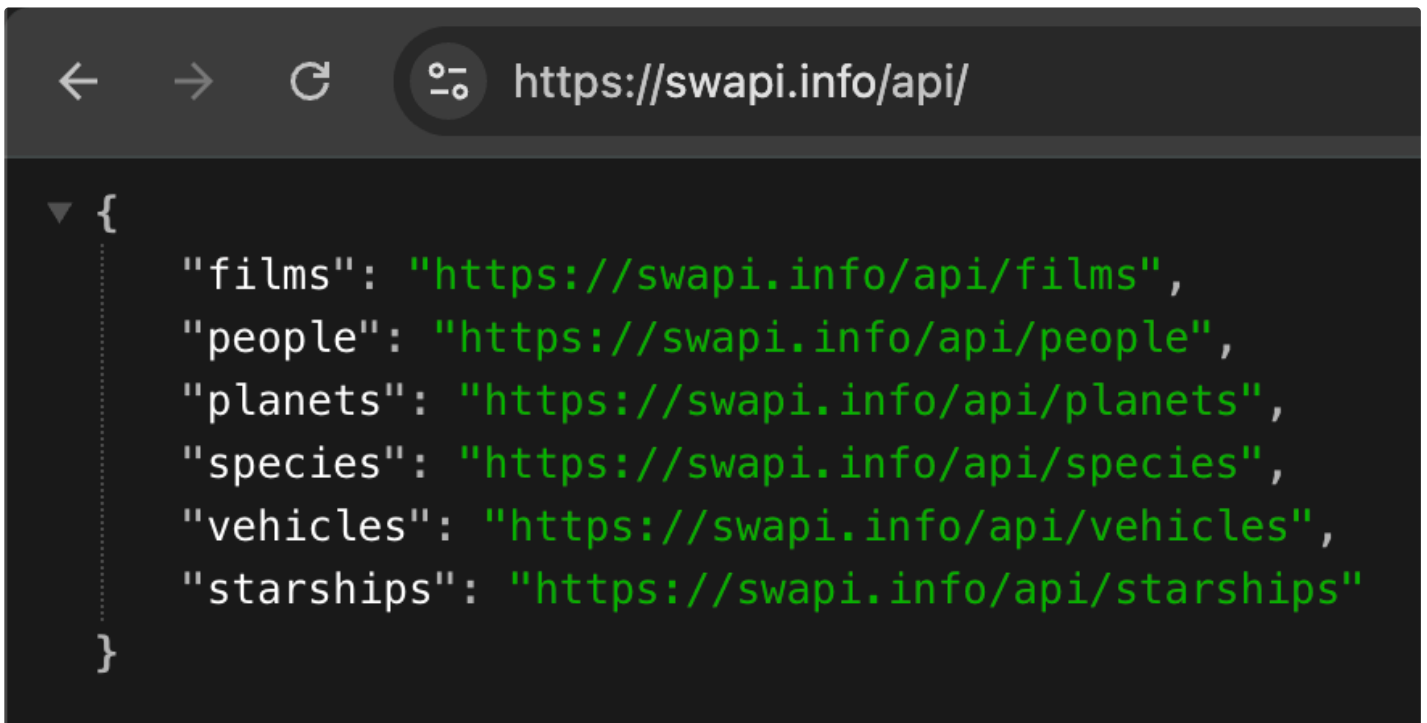


Assignment_5

Task1(a):

A screenshot of a web browser window. The address bar shows the URL "https://swapi.info/api/". Below the address bar, the JSON response is displayed in a dark-themed editor. The JSON is a root object with a single key "films" pointing to a URL, and several other keys pointing to other API endpoints. The JSON is formatted with green text on a dark background.

```
{
  "films": "https://swapi.info/api/films",
  "people": "https://swapi.info/api/people",
  "planets": "https://swapi.info/api/planets",
  "species": "https://swapi.info/api/species",
  "vehicles": "https://swapi.info/api/vehicles",
  "starships": "https://swapi.info/api/starships"
}
```

When hitting the "<https://swapi.info/api/>" API endpoint, we get a JSON response containing a list of all the available API endpoints : films, people, planets, species, vehicles and starships.

1. The first endpoint "<https://swapi.info/api/films/>":

```
▼ [
  ▼ {
    "title": "A New Hope",
    "episode_id": 4,
    "opening_crawl": "It is a period of civil war.\r\nRebel spaceships, striking\r\nfrom a hidden base, have won\r\ntheir first victory against\r\nthe evil Galactic Empire.\r\n\r\nDuring the battle, Rebel\r\nspies managed to steal secret\r\nplans to the Empire's\r\nultimate weapon, the DEATH\r\nSTAR, an armored space\r\nstation with enough power\r\nto destroy an entire planet.\r\n\r\nPursued by the Empire's\r\nsinister agents, Princess\r\nLeia races home aboard her\r\nstarship, custodian of the\r\nstolen plans that can save her\r\npeople and restore\r\nfreedom to the galaxy....",
    "director": "George Lucas",
    "producer": "Gary Kurtz, Rick McCallum",
    "release_date": "1977-05-25",
    ▶ "characters": [ ... ], // 18 items
    ▶ "planets": [ ... ], // 3 items
    ▶ "starships": [ ... ], // 8 items
    ▶ "vehicles": [ ... ], // 4 items
    ▶ "species": [ ... ], // 5 items
    "created": "2014-12-10T14:23:31.880000Z",
    "edited": "2014-12-20T19:49:45.256000Z",
    "url": "https://swapi.info/api/films/1"
  },
  ▶ { ... }, // 14 items
  ▶ { ... }, // 14 items
  ▶ { ... }, // 14 items
  ▶ { ... }, // 14 items
  ▶ { ... } // 14 items
]
Raw Parsed
```

When calling the "films" endpoint we get a list of all the 6 movies, in each one of them there are scalar fields such as 'title' 'episode_id' and some reference fields, each one of them holds a list of API endpoints representing a relationship between the film and the field. As an example, the field 'characters' represents the 'people' that appears in that specific movie.

```
[
  {
    "title": "A New Hope",
    "episode_id": 4,
    "opening_crawl": "It is a period of civil war.\r\nRebel spaceships, striking\r\nfrom a hidden base, have won\r\ntheir first victory against\r\nthe evil Galactic Empire.\r\n\r\nDuring the battle, Rebel\r\nspies managed to steal secret\r\nplans to the Empire's\r\nultimate weapon, the DEATH\r\nSTAR, an armored space\r\nstation with enough power\r\nto destroy an entire planet.\r\n\r\nPursued by the Empire's\r\nsinister agents, Princess\r\nLeia races home aboard her\r\nstarship, custodian of the\r\nstolen plans that can save her\r\npeople and restore\r\nfreedom to the galaxy....",
    "director": "George Lucas",
    "producer": "Gary Kurtz, Rick McCallum",
    "release_date": "1977-05-25",
    "characters": [
      "https://swapi.info/api/people/1",
      "https://swapi.info/api/people/2",
      "https://swapi.info/api/people/3",
      "https://swapi.info/api/people/4",
      "https://swapi.info/api/people/5",
      "https://swapi.info/api/people/6",
      "https://swapi.info/api/people/7",
      "https://swapi.info/api/people/8",
      "https://swapi.info/api/people/9",
      "https://swapi.info/api/people/10",
      "https://swapi.info/api/people/12",
      "https://swapi.info/api/people/13",
      "https://swapi.info/api/people/14",
      "https://swapi.info/api/people/15",
      "https://swapi.info/api/people/16",
      "https://swapi.info/api/people/18",
      "https://swapi.info/api/people/19",
      "https://swapi.info/api/people/81"
    ],
    "planets": [ ... ], // 3 items
    "starships": [ ... ], // 8 items
    "vehicles": [ ... ], // 4 items
    "species": [ ... ], // 5 items
    "created": "2014-12-10T14:23:31.880000Z",
    "edited": "2014-12-20T19:49:45.256000Z",
    "url": "https://swapi.info/api/films/1"
  },
  { ... }, // 14 items
  { ... }, // 14 items
  { ... }, // 14 items
  { ... }, // 14 items
  { ... } // 14 items
]
```

When expanding the 'characters' field, we find each person in the film represented as an API endpoint with the specific id of that person, and in order to get the data of it, you need to call again that specific endpoint.

<u>scalar fields</u>	<u>reference fields</u>
-----------------------------	--------------------------------

- | | |
|--|---|
| <ul style="list-style-type: none">• title• episode_id• director• producer• release_date• created• edited• url | <ul style="list-style-type: none">• characters• planets• starships• vehicles• species |
|--|---|

We categorised 'url' as a scalar field not reference despite having it an API endpoint because it doesn't represent a relationship or connecting a different entity, it points to the object itself.

2. The API endpoint "<https://swapi.info/api/people>":

[illegible]

For the second endpoint we have the same as the first one, some scalar fields alongside some reference one to connect the data.

<u>scalar fields</u>	<u>reference fields</u>
• name • height • mass • hair_color • skin_color • eye_color • birth_year • gender • created • edited • url	• homeworld • films • species • vehicles • starships

3. The API endpoint : "<https://swapi.info/api/planets>":

```
▼ [
  ▼ {
    "name": "Tatooine",
    "rotation_period": "23",
    "orbital_period": "304",
    "diameter": "10465",
    "climate": "arid",
    "gravity": "1 standard",
    "terrain": "desert",
    "surface_water": "1",
    "population": "200000",
    "residents": [
      "https://swapi.info/api/people/1",
      "https://swapi.info/api/people/2",
      "https://swapi.info/api/people/4",
      "https://swapi.info/api/people/6",
      "https://swapi.info/api/people/7",
      "https://swapi.info/api/people/8",
      "https://swapi.info/api/people/9",
      "https://swapi.info/api/people/11",
      "https://swapi.info/api/people/43",
      "https://swapi.info/api/people/62"
    ],
    "films": [
      "https://swapi.info/api/films/1",
      "https://swapi.info/api/films/3",
      "https://swapi.info/api/films/4",
      "https://swapi.info/api/films/5",
      "https://swapi.info/api/films/6"
    ],
    "created": "2014-12-09T13:50:49.641000Z",
    "edited": "2014-12-20T20:58:18.411000Z",
    "url": "https://swapi.info/api/planets/1"
  },
  ▶ { ... }, // 14 items
  ▶ { ... }, // 14 items
  ▶ { ... }, // 14 items
  ▶ { ... }, // 14 items

```

<u>scalar fields</u>	<u>reference fields</u>
• name • rotation_period	• residents • films

- | | |
|---|--|
| <ul style="list-style-type: none">• orbital_period• diameter• climate• gravity• terrain• surface_water• population• created• edited• url | |
|---|--|

4. The API endpoint "<https://swapi.info/api/species>":


```

▼ [
  ▼ {
    "name": "Human",
    "classification": "mammal",
    "designation": "sentient",
    "average_height": "180",
    "skin_colors": "caucasian, black, asian, hispanic",
    "hair_colors": "blonde, brown, black, red",
    "eye_colors": "brown, blue, green, hazel, grey, amber",
    "average_lifespan": "120",
    "homeworld": "https://swapi.info/api/planets/9",
    "language": "Galactic Basic",
    ▼ "people": [
      "https://swapi.info/api/people/66",
      "https://swapi.info/api/people/67",
      "https://swapi.info/api/people/68",
      "https://swapi.info/api/people/74"
    ],
    ▼ "films": [
      "https://swapi.info/api/films/1",
      "https://swapi.info/api/films/2",
      "https://swapi.info/api/films/3",
      "https://swapi.info/api/films/4",
      "https://swapi.info/api/films/5",
      "https://swapi.info/api/films/6"
    ],
    "created": "2014-12-10T13:52:11.567000Z",
    "edited": "2014-12-20T21:36:42.136000Z",
    "url": "https://swapi.info/api/species/1"
  },
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items
  ▶ { ... }, // 15 items

```

Raw

Parsed

scalar fields

reference fields

- name
- classification
- designation
- average_height
- skin_colors
- hair_colors
- eye_colors
- average_lifespan
- language
- created
- edited
- url

- homeworld
- people
- films

5. The API endpoint "<https://swapi.info/api/vehicles>" :

```
▼ [
  ▼ {
    "name": "Sand Crawler",
    "model": "Digger Crawler",
    "manufacturer": "Corellia Mining Corporation",
    "cost_in_credits": "150000",
    "length": "36.8 ",
    "max_atmosphering_speed": "30",
    "crew": "46",
    "passengers": "30",
    "cargo_capacity": "50000",
    "consumables": "2 months",
    "vehicle_class": "wheeled",
    "pilots": [],
    ▼ "films": [
      "https://swapi.info/api/films/1",
      "https://swapi.info/api/films/5"
    ],
    "created": "2014-12-10T15:36:25.724000Z",
    "edited": "2014-12-20T21:30:21.661000Z",
    "url": "https://swapi.info/api/vehicles/4"
  },
  ▶ { ... }, // 16 items
  ▶ { ... }, // 16 items
  ▶ { ... }, // 16 items
  ▶ { ... }, // 16 items
  f ... // 16 items

```

<u>scalar fields</u>	<u>reference fields</u>
• name • model • manufacturer • cost_in_credits • length • max_atmosphering_speed	• pilots • films

- crew
- passengers
- cargo_capacity
- consumables
- vehicle_class
- created
- edited
- url

6. The API endpoint "<https://swapi.info/api/starships>":

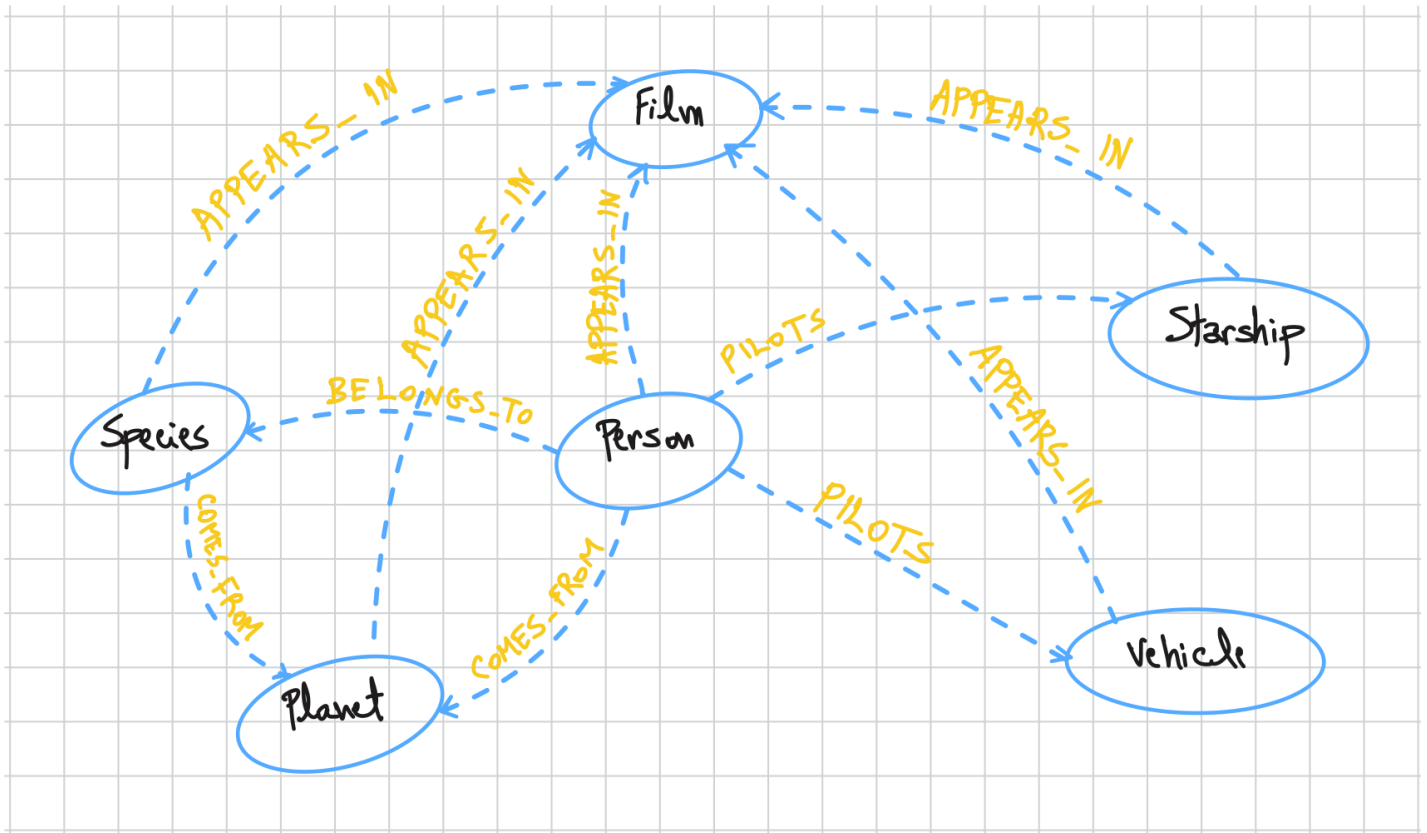
```
▼ [
  ▼ {
    "name": "CR90 corvette",
    "model": "CR90 corvette",
    "manufacturer": "Corellian Engineering Corporation",
    "cost_in_credits": "3500000",
    "length": "150",
    "max_atmosphering_speed": "950",
    "crew": "30-165",
    "passengers": "600",
    "cargo_capacity": "3000000",
    "consumables": "1 year",
    "hyperdrive_rating": "2.0",
    "MGLT": "60",
    "starship_class": "corvette",
    "pilots": [],
    ▼ "films": [
      "https://swapi.info/api/films/1",
      "https://swapi.info/api/films/3",
      "https://swapi.info/api/films/6"
    ],
    "created": "2014-12-10T14:20:33.369000Z",
    "edited": "2014-12-20T21:23:49.867000Z",
    "url": "https://swapi.info/api/starships/2"
  },
  ▶ { ... }, // 18 items
  ▶ { ... }, // 18 items
  ▶ { ... }, // 18 items
  ▶ { ... }, // 18 items

```

<u>scalar fields</u>	<u>reference fields</u>
• name • model • manufacturer • cost_in_credits • length	• pilots • films

- max_atmosphering_speed
- crew
- passengers
- cargo_capacity
- consumables
- hyperdrive_rating
- MGLT
- starship_class
- created
- edited
- url

Task1(b):



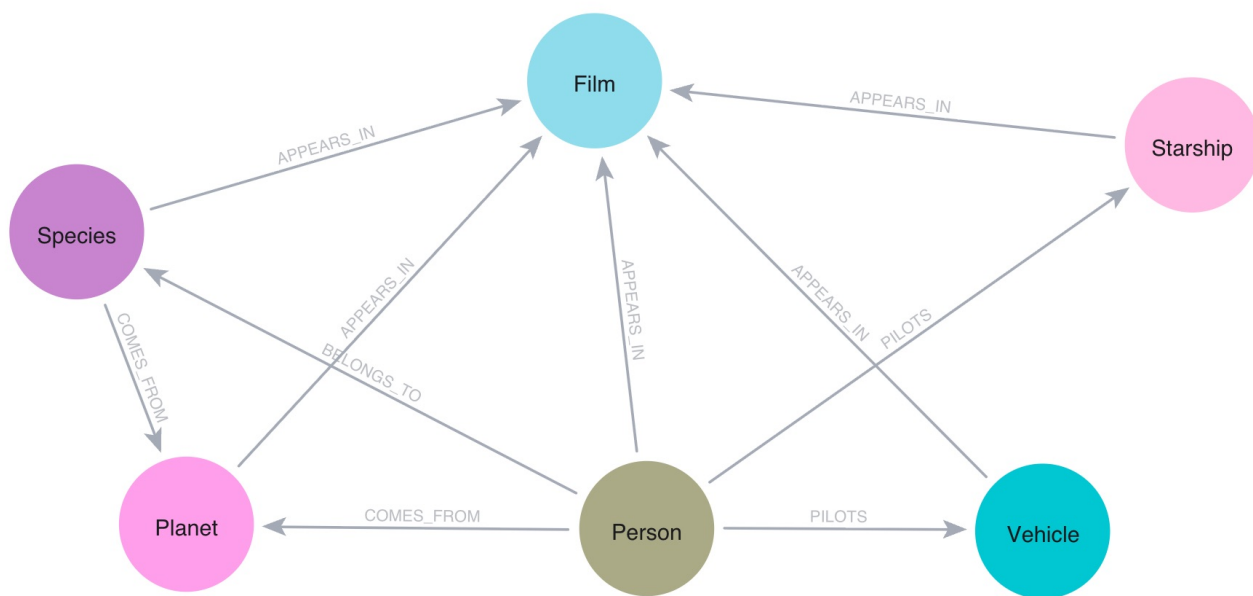
We chose Film, Species, Planet, Person, Starship and Vehicle as the graph nodes because they are basically the 6 API endpoints we have. We are going to keep all the scalar fields as node properties because they describe the characteristics of each node.

For the edges, we converted the reference fields to relationships between the nodes, we kept the naming simple and direct like a logical english sentence (Subject-verb-object), as well as we kept uniform relationship names to reduce schema complexity and make cypher queries easier.

We will take the url field, which we have in every object as an endpoint that has a unique identifier for that object, and make it as the node id. And we will convert all the numerical data from strings to numerical values in the importing phase.

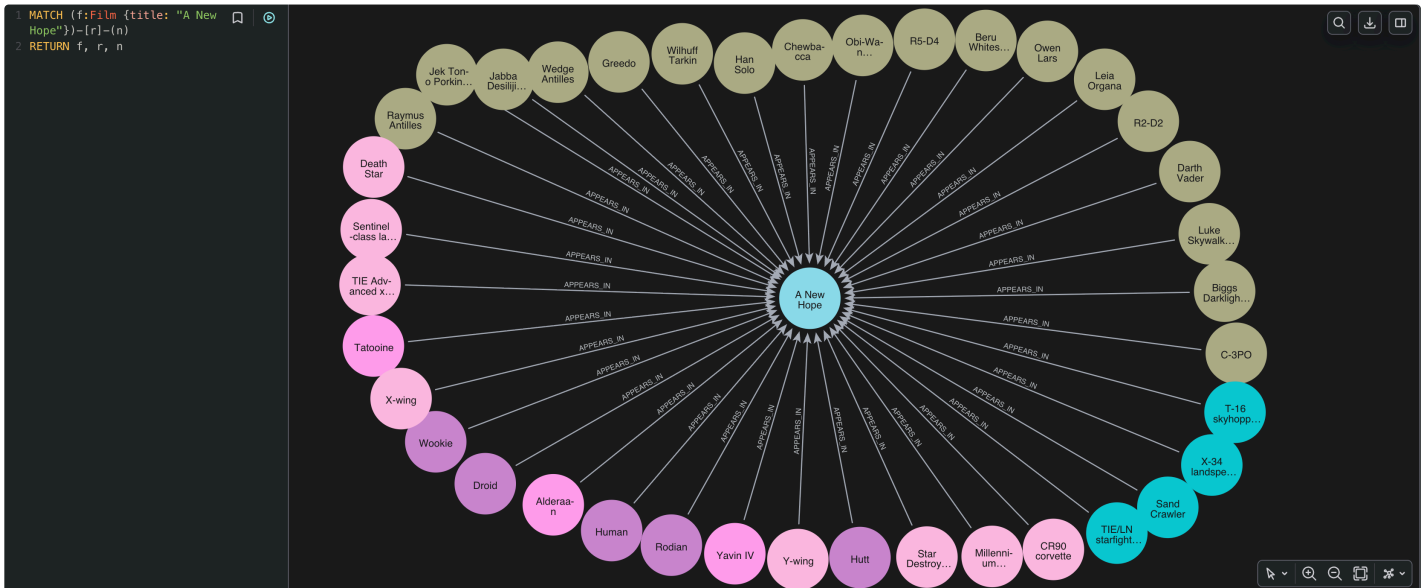
Task1(c):

After getting all the data from the 6 API endpoints and importing it in neo4j graph, we get the following schema that matches perfectly the one we drew before.



We made also a query to get 'APPEARES_IN'

relationship for the movie : " A new Hope".



The full python code with comments is in the file "code.txt"

Task2(a):

In this query we match for the people that appears in a film, we filter the ones that have a non null height, because we already converted the 'unknown' height values to null in the import process. After that, we pass the necessary variables to the next context and we sort them by height descending. Then we group by film, put the sorted characters into a list using "COLLECT" and

get the top 3 tallest characters. Finally, we expand the list again back to individual rows and return "film", "character" and "height".

```
MATCH (p:Person)-[:APPEARS_IN]->(f:Film)
WHERE p.height IS NOT NULL
WITH f.title AS film, p.name AS character, p.height
AS height
ORDER BY height DESC
WITH film, COLLECT({character: character, height:
height})[0..3] AS tallest_characters
UNWIND tallest_characters AS tallest_character
RETURN film, tallest_character.character AS
character, tallest_character.height AS height
```

```
MATCH (p:Person)-[:APPEARS_IN]->(f:Film)
WHERE p.height IS NOT NULL
WITH f.title AS film, p.name AS character, p.height AS height
ORDER BY height DESC
WITH film, COLLECT((character: character, height: height))[0..3] AS tallest_characters
UNWIND tallest_characters AS tallest_character
RETURN film, tallest_character.character AS character, tallest_character.height AS height
```



No graph data available
Your query returned records, but they can't be visualized as a graph.

film	character	height
"The Phantom Menace"	"Yarael Poof"	264
"The Phantom Menace"	"Roos Tarpals"	224
"The Phantom Menace"	"Rugor Nass"	206
"Revenge of the Sith"	"Tarfful"	234
"Revenge of the Sith"	"Chewbacca"	228
"Revenge of the Sith"	"Grievous"	216
"Attack of the Clones"	"Lama Su"	229
"Attack of the Clones"	"Taun We"	213
"Attack of the Clones"	"Ki-Adi-Mundi"	198
"A New Hope"	"Chewbacca"	228
"A New Hope"	"Darth Vader"	202
"A New Hope"	"Raymus Antilles"	188
"The Empire Strikes Back"	"Chewbacca"	228
"The Empire Strikes Back"	"Darth Vader"	202
"The Empire Strikes Back"	"IG-88"	200
"Return of the Jedi"	"Chewbacca"	228

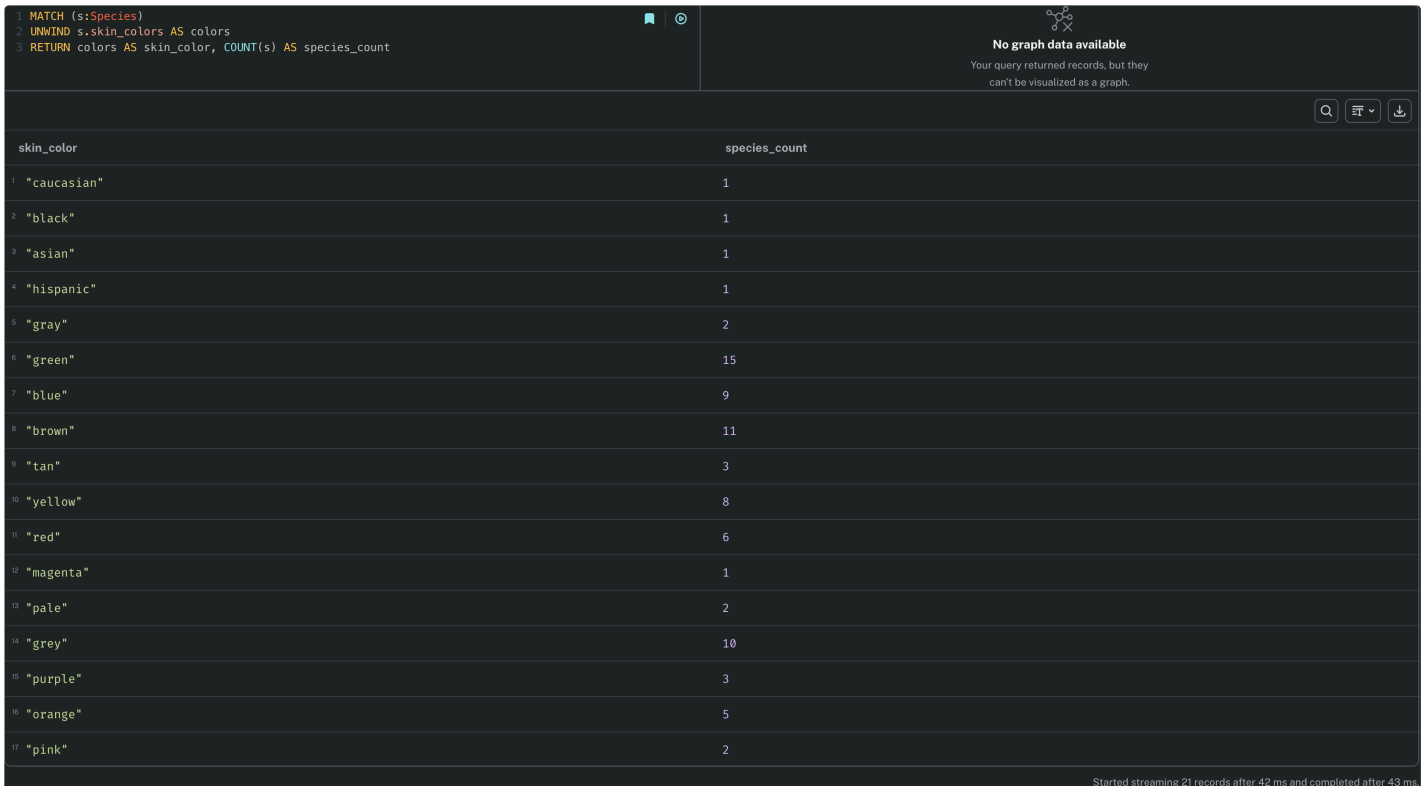
Started streaming 18 records after 5 ms and completed after 7 ms.

Task2(b):

This query is simple because we already stored the "skin_color" as a list, so managing it would be easy. We match all species nodes first, then we expand the list of colors into individual rows. Finally, we return the color and the number of species with that color. We don't need to use distinct here because cypher automatically groups by the color.

MATCH (s:Species)

UNWIND s.skin_colors AS colors
RETURN colors AS skin_color, COUNT(s) AS
species_count



The screenshot shows a Neo4j Cypher query interface. The query is:

```
1 MATCH (s:Species)
2 UNWIND s.skin_colors AS colors
3 RETURN colors AS skin_color, COUNT(s) AS species_count
```

The results table has two columns: `skin_color` and `species_count`. The data is as follows:

skin_color	species_count
"caucasian"	1
"black"	1
"asian"	1
"hispanic"	1
"gray"	2
"green"	15
"blue"	9
"brown"	11
"tan"	3
"yellow"	8
"red"	6
"magenta"	1
"pale"	2
"grey"	10
"purple"	3
"orange"	5
"pink"	2

At the bottom right, a status message reads: "Started streaming 21 records after 42 ms and completed after 43 ms."

Task2(c-i):

In this query we take planets, their residents and the films those residents appear in. Then, we group by the planet and the person to calculate how many films each person appeared in. In the second "WITH" we take the planet, calculate the average and rounding it to 3 decimals, without forgetting the conversion to float in

order to prevent integer division, and finally count the number of people. Finally, we return the results ordered by the "averageFilms" then "planet" in a descending order.

```
MATCH (planet:Planet)<-[:COMES_FROM]-  
(person:Person)-[:APPEARS_IN]->(f:Film)  
WITH planet.name AS planet, person, COUNT(f) AS  
nmbr_films_per_person  
WITH planet,  
ROUND(toFloat(SUM(nmbr_films_per_person))/COU  
NT(person), 3) AS averageFilms, COUNT(person) AS  
residentsCounted  
RETURN planet, averageFilms, residentsCounted  
ORDER BY averageFilms DESC, planet ASC
```

Task2(c-ii):

We use 21 representing the last two digits of the student ID. The result of $21 \bmod 10$ gives 1, and then we add 1 to it based on the assignment's formula, so the final N position will give 2. The planet at position 2 is “Cato Neimoidia” with the exact average films as 3.000 and the number of residents counted as 1.

Task2(c-iii):

The classmate's query "WITH planet, count(f)" calculates the total sum of all film appearances for the planet and put it into one single row, rather than creating multiple rows for each resident like it would if person was included. Because there is only one row per planet, any average calculation will just divide that total by 1. For our assigned planet "Cato Neimoidia", their query coincidentally gives the correct answer of 3.000, but only because the planet has exactly one resident. If it had multiple residents, their query would wrongly output the total sum instead of the true average per resident.

The reason why the query "WITH planet, count(f)" gives the wrong value is that the films are counted just by the planet (no people included), meaning it actually computes the total sum of all movies for that planet into one single row, because of that, we would not know how many residents we have, so the average division will always be divided by one . However, including person in the clause makes sure that we have a row for each resident in that planet to count their movies, giving separated rows that we can use for correct average per resident values.

